

# Task 4: Automated Penetration Testing, Behavioral Analysis & Infrastructure Hardening

Prepared by: Fatimah Adnan

Date: May 2026

Internship ID: APSPL2631283

Timeline: Days 37–48

---

## 1. Executive Summary

The objective of this task was to execute a full-lifecycle security assessment and subsequent hardening of the enterprise test environment. Adhering to the standard Penetration Testing Methodology, the scope progressed systematically through **Reconnaissance** → **Scanning** → **Exploitation** → **Post-Exploitation** → **Reporting & Remediation**. Using a combination of offensive frameworks (Metasploit, Hydra, SET) and defensive controls (UFW, Systemd Auditing), this report documents how legacy service vulnerabilities are uncovered and details the precise engineering steps taken to minimize the network's attack surface.

---

## 2. Penetration Testing Methodology: Reconnaissance & Scanning

Using the database-driven capabilities of the Metasploit Framework, I initialized a background PostgreSQL database (`db_status`) to manage network reconnaissance data and log target vulnerabilities.

### Step 1: Database-Aware Vulnerability Scanning

I executed an aggressive service-detection scan directly through the framework to map out all active daemons on the target VM (`192.168.56.102`).

- **Command:** `db_nmap -sV 192.168.56.102`
- **Findings:** The scan revealed a highly vulnerable attack surface consisting of 23 open TCP ports. The database populated the following critical vectors:

| Port | Protocol | Service | Software Version | Threat Profile |
|------|----------|---------|------------------|----------------|
|      |          |         |                  |                |

|      |     |           |                     |                                                              |
|------|-----|-----------|---------------------|--------------------------------------------------------------|
| 21   | TCP | ftp       | vsftpd 2.3.4        | <b>Critical:</b> Known backdoor payload execution risk.      |
| 22   | TCP | ssh       | OpenSSH 4.7p1       | <b>Medium:</b> Target for credential brute-forcing.          |
| 80   | TCP | http      | Apache httpd 2.2.8  | <b>High:</b> Outdated web server hosting vulnerable scripts. |
| 1524 | TCP | bindshell | Metasploitable Root | <b>Critical:</b> Unauthenticated direct root access.         |
| 6667 | TCP | irc       | UnrealIRCd          | <b>Critical:</b> Remote Code Execution (RCE) backdoor.       |

### 3. Exploitation, Password Attacks & Post-Exploitation

With the reconnaissance data securely logged, I proceeded to simulate multi-angled authentication and software exploits.

#### Step 1: Exploiting the UnrealIRCd Backdoor

I targeted the legacy IRC service running on port 6667, which contains a notorious malicious backdoor implementation.

- **Module:** `exploit/unix/irc/unreal_ircd_3281_backdoor`
- **Payload Config:** Set `RHOSTS 192.168.56.102` and established an automated reverse shell payload.
- **Result:** Successfully bypassed firewall barriers to drop into an active interactive command shell.
- **Post-Exploitation Auditing:** Ran defensive reconnaissance commands within the broken session. Executing `whoami` returned `root`, confirming complete compromise of the underlying operating system.

#### Step 2: Credential Fuzzing & Offline Cracking

To simulate an identity-based attack vector against the administrative infrastructure:

- **Online Brute-Force:** Utilized **Hydra** against the OpenSSH daemon on Port 22 with a targeted dictionary array, successfully isolating valid user login pairs.
- **Offline Password Cracking:** Extracted the target system's hashed password files (**/etc/shadow**) via the root shell. Utilized **John the Ripper** to run the hashes against a wordlist, cracking weak credentials and highlighting structural password policy deficiencies.

---

## 4. Social Engineering Phishing Simulation

To assess organization-wide defensive awareness against client-side exploitation, I transitioned to human-centric threat simulation.

- **Simulation Setup:** Launched the Social-Engineer Toolkit (**SET**) to execute a Credential Harvester Attack.
- **Execution:** Cloned a secure corporate login interface and hosted it via a local web server to capture form submissions.
- **Captured Metrics:** The simulation successfully intercepted mock user credentials in plain text as they traveled through the proxy interface.
- **Remediation & Awareness Training:** Documented an employee awareness brief highlighting visual phishing indicators, such as anomalous URLs, lack of valid cryptographic certificates (HTTPS), and unexpected external authentication prompts.

---

## 5. Malware Basics: Behavioral & Artifact Analysis

To establish a baseline for identifying unknown threats on enterprise endpoints, I performed binary auditing using static and dynamic analysis.

### Step 1: Static Auditing

I audited the binary signature of the local **/usr/bin/whoami** executable to search for embedded malicious anomalies.

- **Command:** **strings /usr/bin/whoami**
- **Observations:** Found standard system libraries (**libc.so.6**), compiler information, and standard help output text. No obfuscated strings, dynamic DNS domains, or packing signatures were detected, verifying the file structure as **benign**.

### Step 2: Dynamic Runtime Tracking

I monitored the execution path of the binary in an isolated sandbox environment to observe system interaction.

- **Command:** **strace whoami**
- **Observations:** Logged all low-level system calls. The output showed normal file openings (**openat**), memory mapping (**mmap**), and terminal printouts before exiting

gracefully with a status code of 0. No unauthorized network sockets or unexpected privilege escalations were initiated.

---

## 6. Defensive Infrastructure Hardening & Remediation

Following the offensive audit, I applied structural system updates to isolate the machine and permanently minimize potential entry points.

### Step 1: Firewall Implementation (UFW)

I transitioned the host into a strict, secure network posture by setting up the Uncomplicated Firewall.

- **Commands:**
- Bash

```
sudo ufw default deny incoming
sudo ufw allow 22/tcp
sudo ufw allow 80/tcp
sudo ufw enable
```

- 
- 
- **Verification:** Running `sudo ufw status` confirmed an active "Default Deny" policy. Essential administrative pathways (SSH and HTTP) are explicitly allowed, while dangerous legacy elements like port 6667 are fully shielded.

### Step 2: Service Minimization (Attack Surface Reduction)

I audited all persistent boot services using `systemctl list-unit-files --state=enabled` to locate legacy or unneeded background daemons.

- **Action:** Found non-essential network dependencies running out-of-the-box, such as `ModemManager.service`.
- **Remediation Commands:**
- Bash

```
sudo systemctl stop ModemManager
sudo systemctl disable ModemManager
```

- - 
  - **Final Hardening Status:** Running a final check via `systemctl list-units --type=service --state=running` verified that only core system infrastructure and security daemons (`haveged` for system entropy generation) remain running.
-

## 7. Conclusion

The comprehensive testing and auditing cycle for Days 37–48 is complete. I have successfully simulated a complete threat matrix—mapping an enterprise attack surface with Metasploit, executing defensive static/dynamic malware reviews, and constructing a hardened server baseline. All accompanying scan XML artifacts, terminal state verifications, and the 10-minute technical demonstration overview have been compiled for final review.

---